

第13章：底层程序设计

目标：

1. 理解计算机中常见的2进制、8进制、10进制、16进制
2. 掌握进制之间的转换的计算
3. 学会原码、反码、补码和计算转化规则
4. 掌握大小端字节序和判断
5. 了解浮点数的存储规则
6. 掌握C语言中的移位操作符和位运算操作符，及使用场景

1. 二进制和进制转换

其实我们经常能听到 2进制、8进制、10进制、16进制 这样的讲法，那是什么意思呢？

其实2进制、8进制、10进制、16进制是数值的不同表示形式而已。

比如：数值15的各种进制的表示形式：

代码块

```
1  15的2进制：1111
2  15的8进制：17
3  15的10进制：15
4  15的16进制：F
5
6  //16进制的数值之前写：0x
7  //8进制的数值之前写：0
```

我们重点介绍一下二进制：

首先我们还是得从10进制讲起，其实10进制是我们生活中经常使用的，我们已经形成了很多常识：

- 10进制中满10进1
- 10进制的数字每一位都是0~9的数字组成

其实二进制也是一样的

- 2进制中满2进1

- 2进制的数字每一位都是0~1的数字组成

那么 1101 就是二进制的数字了。

1.1 2进制转10进制

其实10进制的123表示的值是一百二十三，为什么是这个值呢？其实10进制的每一位是有**权重**的，10进制的数字从右向左是个位、十位、百位....，分别每一位的权重是 $10^0, 10^1, 10^2 \dots$

如下图：

	百位	十位	个位	
10进制的位	1	2	3	
权重	10^2	10^1	10^0	
权重值	100	10	1	
求值	$1 \times 100 + 2 \times 10 + 3 \times 1 =$			123

10进制123每一位权重的理解

2进制和10进制是类似的，只不过2进制的每一位的权重，从右向左是: $2^0, 2^1, 2^2 \dots$

如果是2进制的1101，该怎么理解呢？

2进制的位	1	1	0	1	
权重	2^3	2^2	2^1	2^0	
权重值	8	4	2	1	
求值	$1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 =$				13

2进制1101每一位权重的理解

1.1.1 10进制转2进制数字

2								125	余数为1
	2							62	余数为0,
		2						31	余数为1,
			2					15	余数为1,
				2				7	余数为1,
					2			3	余数为1,
						2		1	余数为1
								0	
10进制的125转换的2进制: 1111101									
10进制转2进制									

由下往上依次所得的余数就是10进制转换出的2进制

1.2 2进制转8进制和16进制

1.2.1 2进制转8进制

8进制的数字每一位是 0~7 的，0~7 的数字，各自写成2进制，最多有3个2进制位就足够了，比如7的二进制是111，所以在2进制转8进制数的时候，从2进制序列中右边低位开始向左每3个2进制位会换算一个8进制位，剩余不够3个2进制位的直接换算。

如：2进制的 01101011，换成8进制：0153，0开头的数字，会被当做8进制。

2进制	0	1	1	0	1	0	1	1
8进制	1			5			3	

1.2.2 2进制转16进制

16进制的数字每一位是0~9, a~f 的，0~9, a~f 的数字，各自写成2进制，最多有4个2进制位就足够了，比如 f 的二进制是1111，所以在2进制转16进制数的时候，从2进制序列中右边低位开始向左每4个2进制位会换算一个16进制位，剩余不够4个二进制位的直接换算。

如：2进制的01101011，换成16进制：0x6b，16进制表示的时候前面加0x

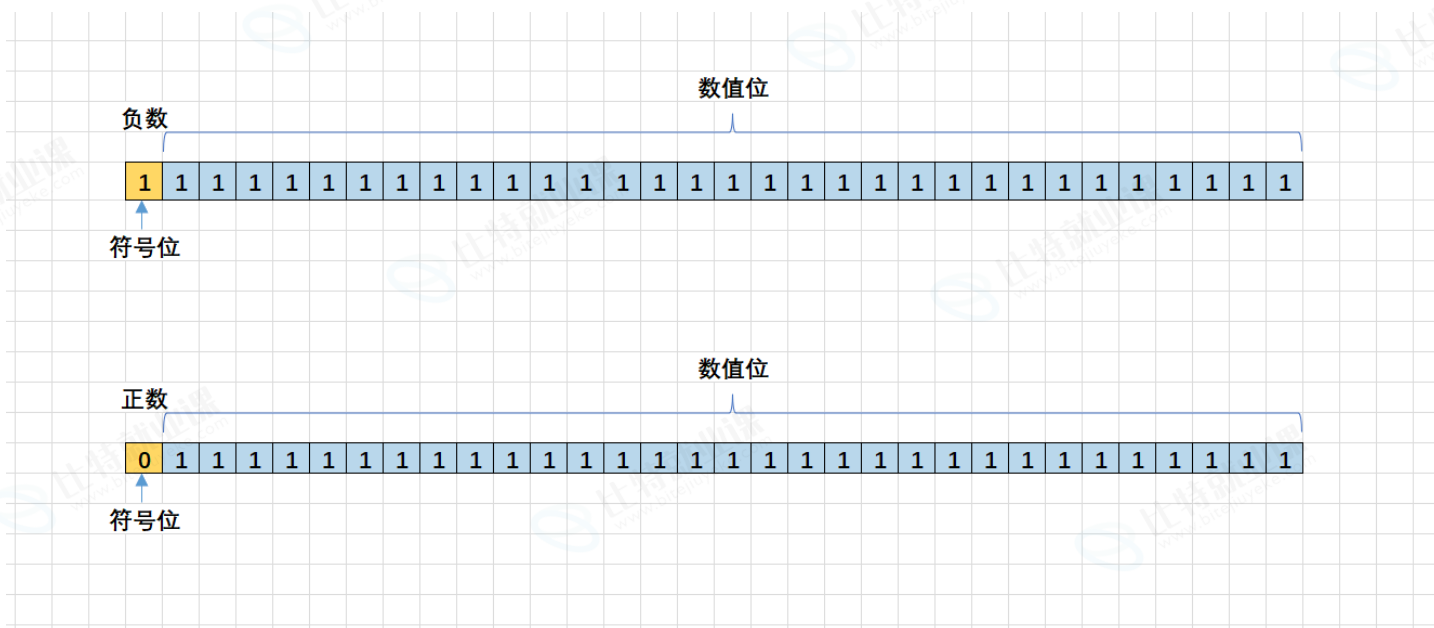
2进制	0	1	1	0	1	0	1	1
16进制	6				b			

2. 原码、反码、补码

整数的2进制表示方法有三种，即原码、反码和补码

2.1 有符号整数

有符号整数的三种表示方法均有**符号位**和**数值位**两部分，2进制序列中，最高位的1位是被当做符号位，剩余的都是数值位。符号位都是用0表示“正”，用1表示“负”。



正整数的原、反、补码都相同。

负整数的三种表示方法各不相同，需要进行计算(方法如下)

原码：直接将数值按照正负数的形式翻译成二进制得到的就是原码。

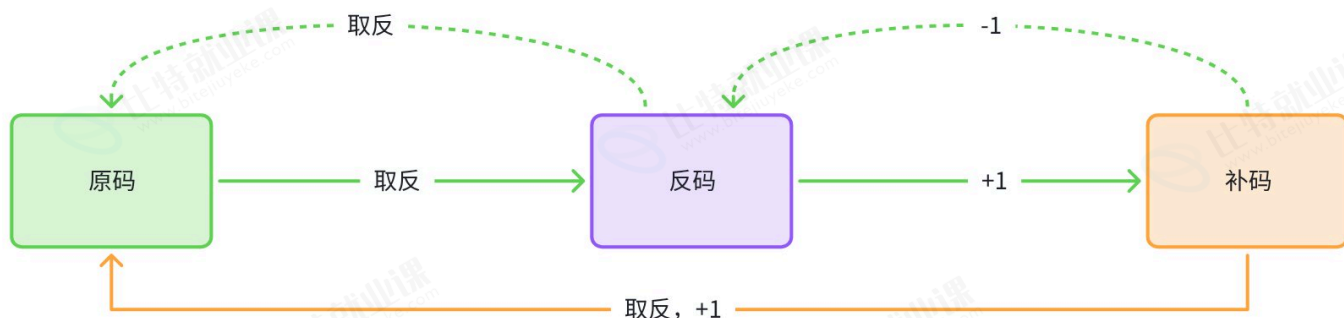
反码：将原码的符号位不变，其他位依次按位取反就可以得到反码。

补码：反码+1就得到补码。

📌 有了补码怎么算出原码呢？有2种方法

1. 原路返回，补码-1，再取反，得到原码
2. 补码取反，+1，也能得到原码

所以原码->补码，补码->原码 的计算方法是可以统一的。



无符号整数的三种 2 进制表示相同，没有符号位，每一位都是数值位。



 整数存在内存中其实存放的是二进制的补码

为什么呢？

在计算机系统中，数值一律用补码来表示和存储。

原因在于，使用补码，可以将符号位和数值域统一处理；同时，加法和减法也可以统一处理（CPU 只有加法器）此外，补码与原码相互转换，其运算过程是相同的，不需要额外的硬件电路。

3. 大小端字节序

当我们了解了整数在内存中存储后，我们借助于调试观察整数在内存中存储的一个细节

代码块

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 0x11223344;
6
7      return 0;
```

调试的时候，我们可以看到在a中的 `0x11223344` 这个数字是按照字节为单位，倒着存储的。这是为什么呢？



内存中存储数据的最小单元是1个字节，计算机内存寻址的最小单位也是字节。所以超过一个字节的数在内存中存储的时候，就有存储顺序的问题，按照不同的存储顺序，我们分为大端字节序存储和小端字节序存储。

小端字节序（存储）模式：

指数据的低位字节内容保存在内存的低地址处，而数据的高位字节内容，保存在内存的高地址处。

		44	33	22	11	
	低地址					高地址

在我们当前的环境下，我们观察到数据就是以小端字节序存储的；

小端字节序存储，看起来是“倒着”的，但更符合CPU做加法的自然过程，CPU做加法时，是从最低位开始，并处理进位。如果采用小端序，最低位字节正好存放在最低地址，CPU可以直接拿着最低地址开始算，无需先跳转到高位。小端字节序存储简化CPU硬件设计，提升计算效率。

这种高效性使得小端字节序成为绝大多数主流CPU的选择，例如Intel x86、x86-64和ARM的默认模式（ARM也支持大端，但通常跑小端）。

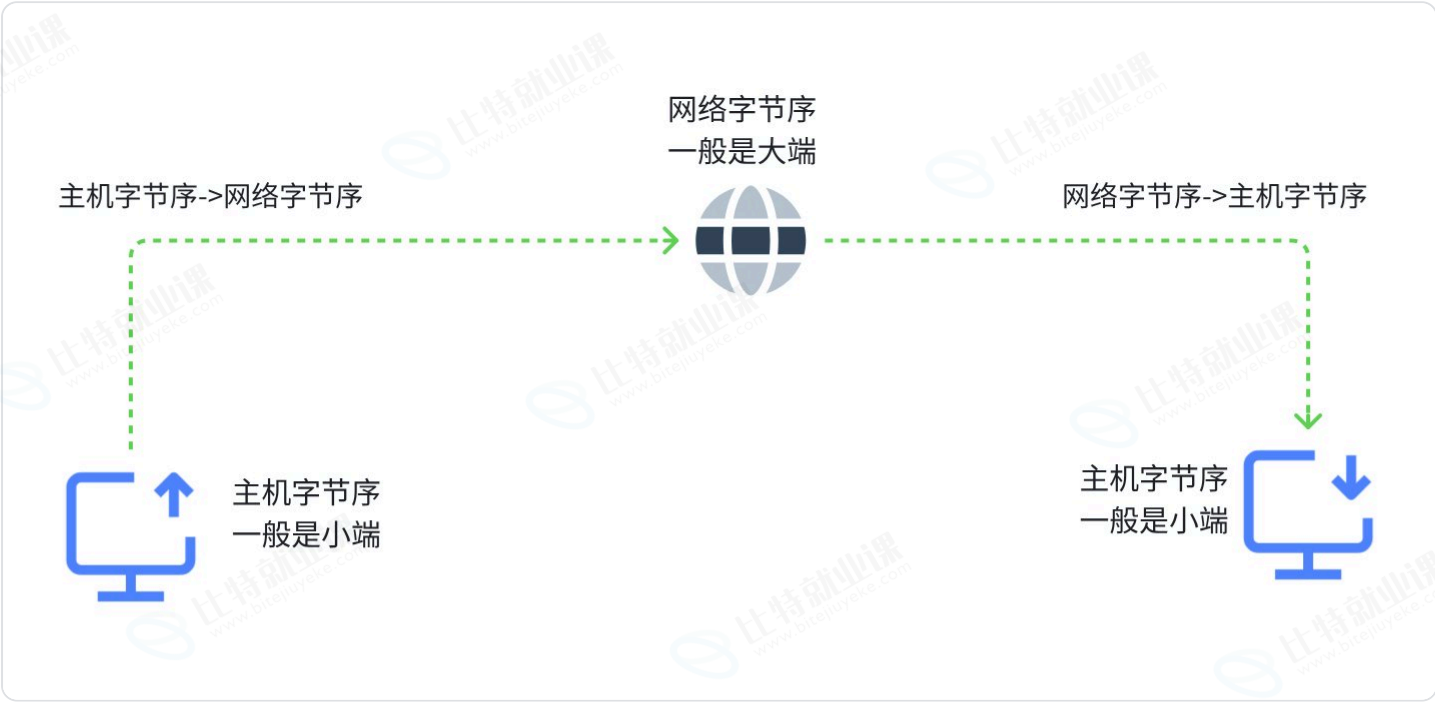
大端字节序（存储）模式：

指数据的低位字节内容保存在内存的高地址处，而数据的高位字节内容，保存在内存的低地址处。

		11	22	33	44	
	低地址					高地址

人类阅读数字是从左到右、高位到低位，所以大端字节序直观、符合人类思维。

网络字节序：为了避免不同计算机之间数据传输时发生混乱，TCP/IP协议族规定大端序为“网络字节序”。当数据在网络上传输时，发送方计算机会将数据从其主机字节序转换成网络字节序（大端），接收方再将其从网络字节序转换回自己的主机字节序。



3.1 练习：大小端的判断

请简述大端字节序和小端字节序的概念，设计一个小程序来判断当前机器的字节序。（10分）

百度笔试题

```

代码块
1  #include <stdio.h>
2
3  int check_sys()
4  {
5      int i = 1;
6      return (*(char *)&i); //取出i所占4个字节中的第一个字节
7  }
8
9  int main()
10 {
11     int ret = check_sys();

```

```

12     if(ret == 1)
13     {
14         printf("小端\n");
15     }
16     else
17     {
18         printf("大端\n");
19     }
20     return 0;
21 }

```

4. 浮点数的存储（录播加餐）

📖 加餐-浮点数的存储

5. 移位操作符

移位操作符的操作数只能是整数，也就是只能对整数进行移位。移位操作符有2个：

操作符	名称	形式	约束	解释
<<	左移操作符	$a \ll b$	b必须 ≥ 0 且 $<$ 左操作数提升后的宽度（否则未定义行为）	将 a 的二进制位(补码)向左移动 b 位
>>	右移操作符	$a \gg b$		将 a 的二进制位(补码)向右移动 b 位

5.1 左移操作符

 **移位规则：**左边抛弃、右边补0

代码块

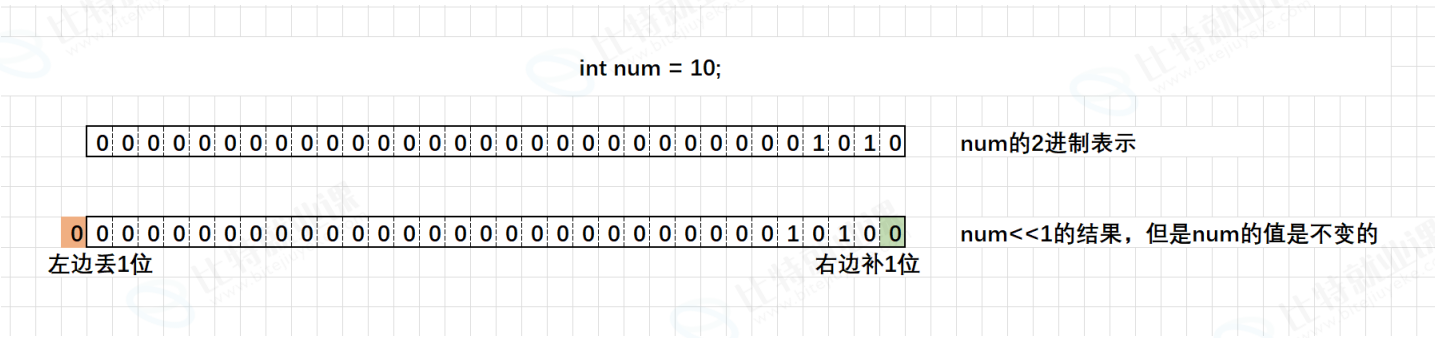
```
1  a << b
```

左操作数a的类型	条件	行为	标准定义
<code>unsigned</code> 类型	任意	低位补0，结果 $\equiv a \times 2^b \pmod{a \text{ 的类型最大取值}+1}$	✓ 完全定义

signed 类型	$a \geq 0$ 且 $a \ll b$ 不溢出	结果为 $a \times 2^b$	✓ 完全定义
signed 类型	$a \geq 0$ 但 $a \ll b$ 溢出	未定义行为	✗ 未定义
signed 类型	$a < 0$	未定义行为	✗ 未定义

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      int num = 10;
5      int n = num << 1;
6      printf("n = %d\n", n);
7      printf("num = %d\n", num);
8      return 0;
9  }
```



左移操作符演示

5.2 右移操作符

- 📌 移位规则：首先右移运算分两种：
- 1. 逻辑右移：左边用0填充，右边丢弃
 - 2. 算术右移：左边用原该值的符号位填充，右边丢弃

代码块

```
1  a >> b
```

左操作数a的类型	条件	行为	标准定义

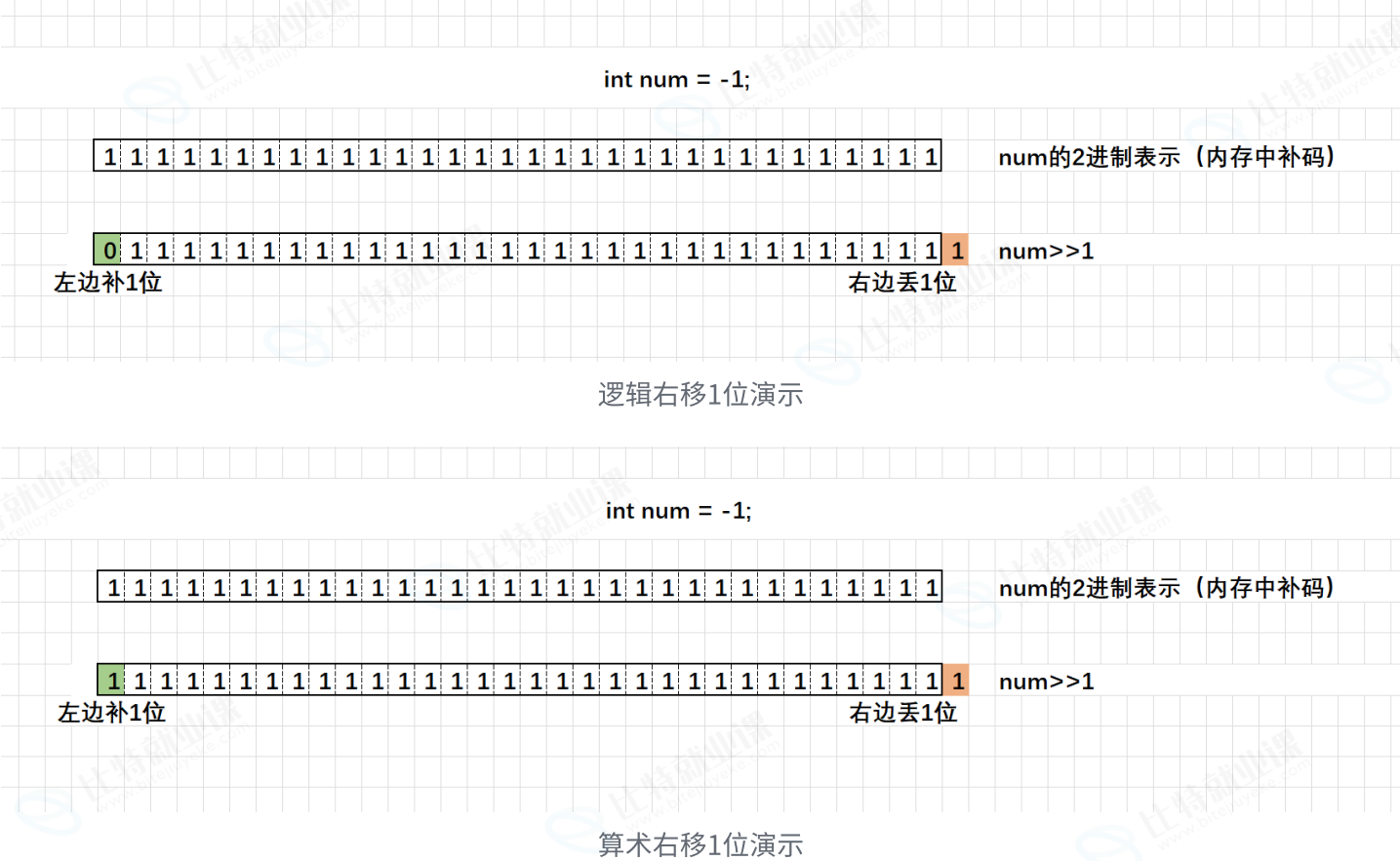
unsigned 类型	任意	算术右移（高位补0），结果 = floor(a / 2^b)，向下取整	✓ 完全定义
signed 类型	a ≥ 0	算术右移（高位补0），结果 = floor(a / 2^b)，向下取整	✓ 完全定义
signed 类型	a < 0	实现定义（通常是算术右移，高位补符号位）	⚠ 实现定义

代码演示：

代码块

```
1  #include <stdio.h>
2
3  int main()
4  {
5      unsigned int num = 10;
6      unsigned int n = num >> 1;
7      printf("n = %u\n", n);
8      printf("num = %u\n", num);
9      return 0;
10 }
```

画图演示：



警告⚠️:

1. 对于移位运算符，不要移动负数位，这个是标准未定义的；
2. 移动的位数取值范围 `[0, type_width - 1]`，`type_width` 是数据类型的2进制位宽度

代码块

```
1  int num = 10;
2  num >> -1; //这样写是错误的，这样的代码不同编译器没有统一的执行方法和结果。
```

最佳实践：左移和右移都优先使用无符号类型

如果是有符号数，可以存储到无符号变量中，再移位

代码块

```
1  int x = -8;
2  unsigned int ux = (unsigned int)x;
3  ux >>= 1;           // 确定的逻辑右移
```

6. 位操作符：&、|、^、~

操作符	名称	形式	计算机规则
&	按位与	$a \& b$	对应二进制位，有0就是0，同时为1才为1
	按位或	$a b$	对应二进制位，有1就是1，同时为0才为0
^	按位异或	$a \wedge b$	对应二进制位，相同为0，相异为1
~	按位取反	$\sim a$	所有二进制位，逐个取反，0变1，1变0

注：位操作符的操作数必须是整数。

按位与：&

代码块

```
1  #include <stdio.h>
2  int main()
3  {
```


异或： $\wedge$

[illegible]

按位取反: ~

```
代码块
1  #include <stdio.h>
2  int main()
3  {
4      printf("%d\n", ~0);
5      //00000000000000000000000000000000 补码
6      //按位取反后
7      //11111111111111111111111111111111 补码
8      //10000000000000000000000000000000
9      //100000000000000000000000000000001 原码
```

```
10     return 0;
11 }
```

6.1 练习1

不能创建临时变量（第三个变量），实现两个整数的交换。

方法1

```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 10;
5      int b = 20;
6      int c = 0;
7      c = a;
8      a = b;
9      b = c;
10     printf("a=%d b=%d\n", a, b);
11     return 0;
12 }
```

方法2

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 10;
6      int b = 20;
7      a = a + b;
8      b = a - b;
9      a = a - b;
10     printf("a=%d b=%d\n", a, b);
11     return 0;
12 }
```

方法1能解决问题，但是创建了c这个临时变量，不满足要求。

方法2中如果a和b的太大，存在溢出的风险，所以有缺陷。

方法3

```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 10;
5      int b = 20;
6      a = a ^ b;
7      b = a ^ b;
8      a = a ^ b;
9      printf("a = %d b = %d\n", a, b);
10     return 0;
11 }
```

异或运算符的特点：

1. $a \wedge a = 0$ （两个相同的数字异或是0）
2. $a \wedge 0 = a$ （任何数字和0异或还是它自己）
3. $a \wedge b \wedge a = a \wedge a \wedge b$ （异或支持交换律）

代码解释

```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 10;
5      int b = 20;
6      a = a ^ b; //a' = a ^ b
7      b = a ^ b; //b = a' ^ b = a ^ b ^ b = a
8      a = a ^ b; //a = a' ^ b = a ^ b ^ b = a ^ b ^ a = b
9      printf("a = %d  b = %d\n", a, b);
10     return 0;
11 }
```

方法3是正确的，而且不会有溢出。

 注意：

方法3虽然正确，但是在实际的开发过程中，交换2个变量的值，通常会采用方法1
方法1的代码可读性好，现代编译器会对此进行优化，性能不会有损失。

6.2 练习2

编写代码实现：求一个整数存储在内存中的二进制中1的个数

方法1

```
1  #include <stdio.h>
2  int main()
3  {
4      int num = 10;
5      int count = 0; //计数
6      while(num)
7      {
8          if(num % 2 == 1)
9              count++;
10         num = num / 2;
11     }
12     printf("1的个数 = %d\n",
13         count);
14     return 0;
15 }
```

这种方法针对负数就失效了

方法2

```
1  #include <stdio.h>
2  int main()
3  {
4      int num = -1;
5      int i = 0;
6      int count = 0; //计数
7      for(i = 0; i < 32; i++)
8      {
9          if( num & (1 << i) )
10             count++;
11     }
12     printf("1的个数 = %d\n",
13         count);
14     return 0;
15 }
```

这种方法不管有几个1，一定会循环32次


```
1 方法3 #include <stdio.h>
2  int main()
3  {
4      int num = -1;
5      int i = 0;
6      int count = 0; //计数
7      while(num)
8      {
9          count++;
10         num = num & (num - 1);
11     }
12     printf("二进制中1的个数 = %d\n", count);
13     return 0;
14 }
```

方法3中使用了Brian Kernighan 算法，它的特点是，有几个1，就循环几次，算法比较高效。

Brian Kernighan算法是统计二进制中1的个数的经典算法。

? 发散一下

如何判断一个数是不是2的幂次方数?

6.3 练习3

二进制位置0或者置1操作符

编写代码将13的二进制序列的第5位修改为1，然后再改回0

代码块

[illegible]

参考代码：

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 13;
5      a = a | (1 << 4);
```

```

6     printf("a = %d\n", a);
7     a = a & ~(1 << 4);
8     printf("a = %d\n", a);
9     return 0;
10 }

```

? AI提示词:

C语言中 >>、<<、&、|、^、~ 这些位操作符都有什么作用？使用场景有哪些？并且给出代码示例。

仔细阅读下面的代码，并体会C语言位操作符的使用场景。

标志位管理

```

1  #include <stdio.h>
2
3  // 定义权限标志位（每个权限占一个二进制位）
4  #define PERM_READ    (1 << 0) // 0b0001 读权限
5  #define PERM_WRITE   (1 << 1) // 0b0010 写权限
6  #define PERM_EXEC    (1 << 2) // 0b0100 执行权限
7  #define PERM_DELETE  (1 << 3) // 0b1000 删除权限
8
9  int main(void)
10 {
11     unsigned char permission = 0; // 初始没有任何权限
12
13     // ===== 1. 设置标志位 =====
14     permission |= PERM_READ;           // 设置读权限
15     permission |= (PERM_WRITE | PERM_EXEC); // 同时设置写和执行权限
16
17     // ===== 2. 检查标志位 =====
18     if (permission & PERM_READ)
19     {
20         printf("拥有读权限\n");
21     }
22     if (permission & PERM_WRITE)
23     {
24         printf("拥有写权限\n");
25     }
26     if (permission & PERM_EXEC)
27     {
28         printf("拥有执行权限\n");
29     }
30     if (permission & PERM_DELETE)

```

```
31 {
32     printf("拥有删除权限\n");
33 }
34
35 // ===== 3. 清除标志位 =====
36 permission &= ~PERM_WRITE;           // 清除写权限
37
38 // ===== 4. 切换标志位 =====
39 permission ^= PERM_DELETE;           // 切换删除权限 (原来0变1)
40 permission ^= PERM_EXEC;             // 切换执行权限 (原来1变0)
41
42 // ===== 5. 输出最终权限状态 =====
43 printf("\n=== 最终权限状态 ===\n");
44 printf("读权限   : %s\n", (permission & PERM_READ) ? "开启" : "关闭");
45 printf("写权限   : %s\n", (permission & PERM_WRITE) ? "开启" : "关闭");
46 printf("执行权限: %s\n", (permission & PERM_EXEC) ? "开启" : "关闭");
47 printf("删除权限: %s\n", (permission & PERM_DELETE) ? "开启" : "关闭");
48
49 return 0;
50 }
```